



۸۱۰۱۰۱۴۹۲



تمرین ۴

شبکه‌های کامپیوتری
نام و نام خانوادگی: علیرضا کریمی

۱

۱.۱

(آ): طبق چیزی که از جمینای [۱] و چت‌جی‌پی‌تی [۲] پرسیدم، گفتند که وقتی که با توجه به *longest prefix match* باز هم به چند تا *interface* و پورت مختلف رسیدیم، در مراحل بعدی طبق معیار های دیگری این جداسازی و انتخاب *interface* را انجام دهیم. این موارد عبارتند از

- *administrative distance/performance*: اینکه وقتی برای رسیدن به مقصد از چندین مسیر میتوانیم برویم، به بعضی از مسیر ها بیشتر از مسیر های دیگر و یا بعضی پروتکل ها بیشتر از پروتکل های دیگر، اولویت میدهیم. مثلا دو مسیر با دو پروتکل مختلف داریم و اونی را که اولیت بالاتری دارد، انتخاب میکنیم.
- *Metrics*: مثلا اگر هر دو مسیر یک پروتکل مسیر یابی استفاده کرده باشند، آنوقت مثلا مسیری که هزینه کمتری (یا طبق یک متریک دیگر) دارد را انتخاب میکنیم.
- *load balancing*: اگر همه این موارد یکی بودند، میتوانیم لود را بین مسیر های مختلف پخش کنیم که به قول جمینای به آن *ECMP* میگوید.

(ب): طبق ریسورس های قبل [۱] و [۲]، با حذف *otherwise* یعنی هر پیامی که مقصدش در جدول پوشش داده نشده *drop* میشود و به مقصد نمیرسد. این به نوعی باعث میشود که آن مقصد از دید فرستنده به صورت *unreachable* تلقی شود.

در واقع خود روتر هم وقتی بسته را میبیند که *prefix* آن میچ نمیشود، آن را *drop* میکند و یک پیغام *ICMP Destination Unreachable* به سمت فرستنده میفرستد. مثلا همان پیام *net unreachable*

۲.۱

(آ): این مورد *prefix* فقط با مورد *interface* شماره ۵ میخواند. پس به همان ارسال میشود.

(ب): اگر به *prefix* آن دقت کنیم، فقط به *interface* شماره ۱ همخوان است و به آن ارسال میشود.

(ج): اینجا هم با *interface* شماره ۱ و هم با شماره ۳ همخوان است و طبق قاعده *longest prefix match* با ۳ همخوان است و به *interface* شماره ۳ ارسال میشود.

۲

اسلاید ۴ درس صفحه ۲۹

۱.۲

(الف): این اصلی الگوریتم *WFQ*: در این الگوریتم با توجه به وزنی که به هر صف داده، نسبت به وزن داده شده به صف های دیگر، تعداد بسته های ارسالی از هر صف متناسب با آن تنظیم میشود. مثلاً به طور نسبی نسبت به یک دیگر، اگر ۵ تا بسته از صف اول برود، ۳ بسته از صف دوم و ۲ بسته از صف سوم. (اون ۵ تا بسته پشت سر هم نمیروند لزوماً) یعنی به صف ۱ اولویت بیشتری داده و میتواند بسته های بیشتری نسبت به صف ۳ ارسال کند اما اینگونه نیست که خروجی صف ۳ را صفر کند. فقط درصد بسته های خروجی از صف های کم اولویت تر را کمتر کرده.

درواقع به ازای هر ۱۰ بسته ای که خارج می شود، انتظار داریم ۵ تا سهم کلاس یک، ۳ تا سهم کلاس دو و ۲ تا سهم کلاس سه باشد.

همانطور که در صفحه ۲۹ اسلاید ۴ گفته شده و از جملینای [۳]، در *priority queue* اولویت همیشه با صف با اولویت بالاتر است و همیشه صف با اولویت بالاتر سرویس میگیرد. اگر در صف با اولویت بالاتر به صورت مداوم ورودی داشته باشیم، همیشه همان صف اجرا میشود و اصلاً داده های صف های با اولویت کمتر ارسال نمیشود. پس با *starvation* روبرو خواهیم بود. اما در *WFQ* به هر کدام از صف ها یک وزنی میدهد که هر چند مدت یک بار حتماً از هر صف، داده ای ارسال شود، اینگونه به هیچ عنوان صد در صد، صف ۱ نمیتواند جلوی صف ۳ را بگیرد و *starvation* نداریم. چون بالاخره با *rate* کمتر، صف ۳ اجرا میشود حداقل مثلاً.

پس از این نظر، منصفانه تر از *priority queue* عمل میکند.

(ب): منطقاً در *WFQ*، *starvation* رخ نمیدهد چون بالاخره یک سهمی برای ارسال داده روی لینک دارد. که بالاخره ارسال میشود. اما در *priority queue* اگر قرار باشد به طور مداوم (به گونه ای که ریت ورودی شون از ریت خروجی شون مساوی یا بیشتر باشه) داده وارد صف ۱ یا ۲ شود، همیشه اولویت با صف های ۱ و ۲ است و هیچگاه به صف ۳ نمیرسد و *starvation* رخ میدهد.

۲.۲

برای محاسبات این بخش از جمینای [۳] کمک گرفتیم. برای هر بسته، ما یک زمان پایان مجازی (F) محاسبه می‌کنیم:

$$F_{i,k} = \max(F_{i,k-1}, a_{i,k}) + \frac{1}{w_i}$$

- i (شماره کلاس): نشان‌دهنده دسته‌بندی ترافیک است (در مسئله ما کلاس ۱، ۲ یا ۳).
- k (شماره بسته در کلاس): نشان می‌دهد این چندمین بسته‌ای است که برای این کلاس خاص وارد شده است.
- $a_{i,k}$ (زمان رسیدن): لحظه‌ای که بسته وارد صف روتر می‌شود.
- $F_{i,k-1}$ (زمان پایان بسته قبلی): زمان پایان مجازی بسته‌ای که دقیقاً قبل از این بسته در همان کلاس بوده است.
- L (طول بسته): اندازه فایل که در این سوال برای همه ۱ فرض شده است.
- w_i (وزن کلاس): سهمی از پهنای باند که به آن کلاس اختصاص داده شده (مثلاً ۵.۰ برای کلاس ۱).

مقادیر $(1/w)$ برای هر کلاس:

کلاس ۱ (وزن ۰.۵): هر بسته ۲ واحد به زمان مجازی اضافه می‌کند ($1/0.5 = 2$).

کلاس ۲ (وزن ۰.۳): هر بسته ۳.۳۳ واحد اضافه می‌کند ($1/0.3 = 3.33$).

کلاس ۳ (وزن ۰.۲): هر بسته ۵ واحد اضافه می‌کند ($1/0.2 = 5$).

و طبق آن فرمول این نتایج بدست می‌آید:

کلاس	شماره بسته	زمان رسیدن a	محاسبات F	زمان پایان مجازی F
1	1	0.37	$0.37 + 2$	2.37
	2	0.73	$2.37 + 2$	4.37
	3	1.03	$4.37 + 2$	6.37
	4	1.38	$6.37 + 2$	8.37
	5	1.77	$8.37 + 2$	10.37
	9	3.29	$10.37 + 2$	12.37
	10	3.64	$12.37 + 2$	14.37
2	6	2.21	$2.21 + 3.33$	5.54
	7	2.70	$5.54 + 3.33$	8.87
	8	2.99	$8.87 + 3.33$	12.20
	14	5.00	$12.20 + 3.33$	15.53
	15	5.65	$15.53 + 3.33$	18.86
3	11	4.04	$4.04 + 5$	9.04
	12	4.32	$9.04 + 5$	14.04
	13	4.68	$14.04 + 5$	19.04

جدول ۱: محاسبه‌ی زمان پایان مجازی F برای بسته‌ها در WFQ

قانون انتخاب: در ابتدای هر اسلات، از بین بسته‌هایی که تا آن لحظه رسیده‌اند، بسته‌ای که کمترین F را دارد انتخاب می‌شود.

تایم صفر که بسته‌ای نداریم، از تایم ۱ می‌توانیم این کار را انجام دهیم. تایم ورودی هر بسته نسبت به تایم F به گونه‌ای است که لازم نیست نگران تایم ورودی بسته‌ها باشیم. در جدول ۲ می‌توانید ببینید که در هر تایم، کمترین F را کدام داشته و انتخاب شده و زمان ورودش هم مشکلی نداشته.

زمان اسلات t	شماره بسته انتخابی	زمان ورود a	زمان پایان مجازی F	وضعیت دسترسی
$t = 1$	بسته 1	0.37	2.37	رسیده ($0.37 < 1$)
$t = 2$	بسته 2	0.73	4.37	رسیده ($0.73 < 2$)
$t = 3$	بسته 6	2.21	5.54	رسیده ($2.21 < 3$)
$t = 4$	بسته 3	1.03	6.37	رسیده ($1.03 < 4$)
$t = 5$	بسته 4	1.38	8.37	رسیده ($1.38 < 5$)
$t = 6$	بسته 7	2.70	8.87	رسیده ($2.70 < 6$)
$t = 7$	بسته 11	4.04	9.04	رسیده ($4.04 < 7$)
$t = 8$	بسته 5	1.77	10.37	رسیده ($1.77 < 8$)
$t = 9$	بسته 8	2.99	12.20	رسیده ($2.99 < 9$)
$t = 10$	بسته 9	3.29	12.37	رسیده ($3.29 < 10$)
$t = 11$	بسته 12	4.32	14.04	رسیده ($4.32 < 11$)
$t = 12$	بسته 10	3.64	14.37	رسیده ($3.64 < 12$)
$t = 13$	بسته 14	5.00	15.53	رسیده ($5.00 < 13$)
$t = 14$	بسته 15	5.65	18.86	رسیده ($5.65 < 14$)
$t = 15$	بسته 13	4.68	19.04	رسیده ($4.68 < 15$)

جدول ۲: جدول انتخاب بسته‌ها در WFQ به همراه وضعیت دسترسی در هر اسلات

ترتیب نهایی ارسال (بر اساس کمترین F در هر لحظه t):

۱ ← ۲ ← ۳ ← ۴ ← ۷ ← ۱۱ ← ۵ ← ۸ ← ۹ ← ۱۲ ← ۱۰ ← ۱۴ ← ۱۵ ← ۱۳

۳

برای حل این سوال از اسلاید چهارم درس فصل سومش و جمینای [۴] کمک گرفتم.

۱.۳

(الف): این آدرس عمومی است چون حزو آدرس های *private* نیست. چون آدرس‌های خصوصی (*Private*) طبق استاندارد RFC 1918 در محدوده‌های مشخصی قرار دارند (مانند 10.0.0.0/8 یا 172.16.0.0/12 یا

192.168.0.0/16. [۴] از آنجایی که آدرس شروع شده با ۲۱۶ در هیچ‌کدام از این محدوده‌ها نیست، آدرس عمومی محسوب شده و در اینترنت قابل مسیریابی است.

(ب): تعداد کل بیت‌ها ۳۲ است. وقتی که ۲۵ بیت برای زیر شبکه از آن جدا می‌کنیم، کلاً ۸ بیت برای فضای آدرس دهی داخلی زیر شبکه باقی مانده است. پس در کل $2^8 = 256$ آدرس خواهیم که البته از این تعداد باید ۲ واحد کم کنیم.

علت کم کردن ۲ آدرس:

- آدرس شبکه (*Network Address*): اولین آدرس در هر محدوده (که تمام بیت‌های بخش میزبان آن ۰ است) برای شناسایی خود زیر شبکه استفاده می‌شود و نمی‌توان آن را به یک دستگاه اختصاص داد.
 - آدرس پخش (*Broadcast Address*): آخرین آدرس در هر محدوده (که تمام بیت‌های بخش میزبان آن ۱ است) برای ارسال داده به تمامی دستگاه‌های موجود در آن زیر شبکه به صورت همزمان رزرو شده است.
- با کم کردن دو آدرس شبکه و آدرس *broadcast*، آدرس‌های قابل استفاده برای هاست‌های آن زیر شبکه $256 - 2 = 254$ خواهد بود.

۲.۳

برای اختصاص بهینه، ابتدا از بزرگترین زیر شبکه (نیاز به بیشترین *Host*) شروع می‌کنیم باید نزدیک‌ترین توان ۲ که بزرگتر از آن تعداد خواسته شده است را در نظر بگیریم. [۴] و اینگونه آدرس *CIDR* برای هر زیر شبکه را تعیین می‌کنیم.

(الف): ۱. زیر شبکه *A* (نیاز به ۱۲۱ میزبان): نزدیک‌ترین توان ۲ که بزرگتر از ۱۲۱ باشد، ۱۲۸ است (2^7). پس به ۷ بیت برای *Host* نیاز داریم. ماسک شبکه: $32 - 7 = 25$. آدرس: 216.135.46.0/25

۲. زیر شبکه *B* (نیاز به ۴۹ میزبان): نزدیک‌ترین توان ۲ که بزرگتر از ۴۹ باشد، ۶۴ است (2^6). پس به ۶ بیت برای *Host* نیاز داریم. ماسک شبکه: $32 - 6 = 26$. آدرس: 216.135.46.128/26

۳. زیر شبکه *C* (نیاز به ۲۱ میزبان): نزدیک‌ترین توان ۲ که بزرگتر از ۲۱ باشد، ۳۲ است (2^5). پس به ۵ بیت برای *Host* نیاز داریم. ماسک شبکه: $32 - 5 = 27$. آدرس: 216.135.46.192/27

همانطور که می‌بینید وقتی که از ۰ تا ۱۲۷ را به زیر شبکه *A* اختصاص دادیم، برای زیر شبکه *B* از ۱۲۸ به بعد که بیت ۸ برای ۱ و بیت ۷ برابر صفر است. برای زیر شبکه *C* هم بیت ۸ و ۷ برابر ۱ و بیت ۶ برابر صفر است و اینگونه تمایز قائل می‌شویم.

قاعداً برای زیرشبکه جدید که بتواند وارد شبکه شود، میتوانیم بیت های ۸ و ۷ و ۶ را یک بگذاریم و پس حداکثر ۵ بیت داریم برای فضای آدرس دهی میزبان های آن (البته آن آدرس های شبکه و *broadcast* آن را هم میتوانیم در نظرش نگیریم).

(ب،ج،د): میتوانید اطلاعات مربوط به آدرس های شبکه و آدرس های *broadcast* هر زیرشبکه که همان آدرس های اول و آخر قابل اختصاص به زیر شبکه هستند، در جدول ۳ ببینید.

زیرشبکه	آدرس شبکه (CIDR)	اولین IP قابل تخصیص	آخرین IP قابل تخصیص	آدرس Broadcast
A	216.135.46.0/25	216.135.46.1	216.135.46.126	216.135.46.127
B	216.135.46.128/26	216.135.46.129	216.135.46.190	216.135.46.191
C	216.135.46.192/27	216.135.46.193	216.135.46.222	216.135.46.223

جدول ۳: اطلاعات زیرشبکه‌ها و بازه آدرس‌های قابل تخصیص

پس با این طراحی، فضای آدرس از مقدار 224 تا 255 (یک بلوک 27) کاملاً آزاد باقی مانده است که می‌تواند برای اضافه شدن یک زیرشبکه جدید در آینده (طبق صورت سوال) استفاده شود [۴]

۴

اسلاید ۴ درس صفحه ۵۶

۱.۴

(الف): چون آدرس گفته شده با 10.0 شروع شده که فکر کنم جزو آدرس های *private* است و برای همین نمیتوان از آن در شبکه اینترنت عمومی و *public* استفاده کرد و اصلاً در آن *routable* نیست. برای همین روتر *NAT* باید آدرس خصوصی را با آدرس عمومی و معتبر خود (*WAN IP*) جایگزین بکند تا امکان ارتباط با دنیای خارج فراهم شود. [۵]

(ب): این روش برای بسته های *outgoing*، یک مدخل در *NAT translation table* درست میکند که آدرس *IP* و پورت داخلی را برای مبدا به آدرس *IP* اون *NAT* و یک پورت جدید *assign* میکند. یعنی اون پورت جدید را میگیرد که کلاً کار های مربوط به این آدرس و پورت داخلی را هندل کند.

اینگونه بسته به مقصد ارسال میشود. حال اگر گیرنده، به همین آدرس *NAT* و همین پورت جدید به عنوان *destination*، پیامی را بفرستد، *NAT router* میاید و به نگاشت در جدول *NAT* نگاه میکند و از روی اون پورت جدیدی که اختصاص داده، متوجه میشود که آدرس *IP* و پورت نود داخلی چی بوده و با آن *replace* میکند. [۵]

۲.۴

طبق صورت سوال، اطلاعات اولیه ما این است:

- میزبان داخلی: 10.0.1.14 با پورت مبدا 3364
- سرور مقصد: 128.119.171.185 با پورت مقصد 80
- آدرس روتر (*WAN*): 135.122.203.205

(الف): بسته پس از ترک میزبان و قبل از رسیدن به روتر:

در این مرحله، بسته هنوز در شبکه داخلی است و تغییری نکرده است.

Source (مبدا): 10.0.1.14 ، *Port*: 3364

Destination (مقصد): 128.119.171.185 ، *Port*: 80

(ب): پس از ارسال توسط روتر به اینترنت:

روتر آدرس مبدا را به آدرس عمومی خود (آدرس *NAT* در سمت *LAN*) تغییر می‌دهد.

Source (مبدا): 135.122.203.205

Destination (مقصد): 128.119.171.185 ، *Port*: 80

آیا پورت مبدا تغییر می‌کند؟ بله، در اکثر پیاده‌سازی‌های *NAT* روتر یک شماره پورت جدید (مثلاً ۵۰۰۱) از پورت‌های آزاد خود مخصوص به آن آدرس *IP* و پورت داخلی اختصاص می‌دهد تا بتواند چندین ارتباط مختلف داخلی را از هم تشخیص دهد.

(ج): پس از بازگشت بسته از اینترنت و قبل از ورود به روتر:

این بسته پاسخی است که از طرف سرور ارسال شده است.

Source (مبدا): 128.119.171.185 ، *Port*: 80

Destination (مقصد): 135.122.203.205 (آدرس عمومی روتر) و پورت جدید اختصاص یافته در مرحله

قبل (مثلاً همان ۵۰۰۱)

(د): پس از ارسال بسته توسط روتر به میزبان داخلی:

روتر با نگاه به جدول، آدرس مقصد را دوباره به آدرس خصوصی برمی‌گرداند. درواقع یک نگاشت بین پورت ۵۰۰۱ و اون آدرس *IP* و پورت داخلی ۳۳۶۴ در مرحله ۲ ایجاد شده بود.

Source (مبدأ): 128.119.171.185، *Port*: 80

Destination (مقصد): 10.0.1.14، *Port*: 3364

آیا ورودی جدیدی در جدول *NAT* ساخته شده است؟ خیر، ورودی جدید در مرحله ۲ (هنگام خروج بسته) ساخته شده بود. در این مرحله (مرحله ۴)، روتر فقط از اطلاعات موجود در جدول استفاده می‌کند تا نگاشت به آدرس *IP* و پورت داخلی نود را انجام دهد و بسته را به مقصد برساند.

۵

اسلاید ۴ درس صفحه ۶۲ و ۶۳ و برای چک کردن جواب از جمینای کمک گرفتیم [۶]

۱.۵

(الف): همانطور که در اسلاید ۴ درس صفحه ۶۳ میبینید، بعضیی از روتر ها به دلیل پشتیبانی نکردن از تکنولوژی جدید *IPv6* تنها از پروتکل *IPv4* استفاده میکنند و تنها هدر های آن را متوجه میشوند. پس در این موارد لازم است که دیتاگرام *IPv6* را به عنوان *payload* در یک دیتاگرام *IPv4* قرار دهیم تا بسته توسط آن روتر های قدیمی قابل مطالعه باشد

(ب): ورودی و خروجی تونل باید از هر دو پروتکل *IPv4* و *IPv6* پشتیبانی کنند. ورودی تولینگ باید بسته *IPv6* را در یک بسته *IPv4*، *encapsulate* کند (به نوعی آدرس مبدأ *IPv4* را خودش میگذارد و آدرس مقصدش را آدرس روتر تونل خروجی میگذارد تا وقتی بدست آن رسید، بسته را از این حالت خارج کند) و برای خروجی تونل هم باید بسته *IPv6* را از داخل بسته *IPv4* جدا کنیم (*decapsulate*) و به ادامه مسیر بدهیم.

۲.۵

(الف): در ارسال از *F* به *b*:

نوع دیتاگرام، *IPv4* (چون داخل تونل است). است. آدرس مبدأ 234.108.233.156 (آدرس *IPv4* روتر *F* که در شروع تونل است) و آدرس مقصد 247.96.164.230 (آدرس *IPv4* روتر *C* که انتهای تونل است). آیا پیام کپسوله شده است؟ بله. چون لازم است که دیتاگرام *IPv6* در *IPv4*، *encapsulate* شود.

(ب): در ارسال از b به a :

نوع دیتاگرام، $IPv4$ است.

آدرس مبدأ: 234.108.233.156 (مبدأ تونل یعنی F)

آدرس مقصد: 247.96.164.230 (مقصد تونل یعنی C)

آدرس‌های دیتاگرام درونی ($IPv6$):

• مبدأ درونی: آدرس میزبان در زیرشبکه D :

9927 : 97D1 : 35B3 : 7B7B : E7EA : 9AAE : EB2B : 8826

• مقصد درونی: آدرس میزبان در زیرشبکه A :

F26C : C9CA : 41CD : F1CF : 44E8 : A4FB : A389 : 2A7A

(ج): در ارسال از C به A :

نوع دیتاگرام: $IPv6$ (چون بسته از تونل خارج شده است)

آدرس مبدأ: آدرس میزبان در زیرشبکه D

9927 : 97D1 : 35B3 : 7B7B : E7EA : 9AAE : EB2B : 8826

آدرس مقصد: آدرس میزبان در زیرشبکه A

F26C : C9CA : 41CD : F1CF : 44E8 : A4FB : A389 : 2A7A

آیا کپسوله شده است؟ خیر، چون کپسول $IPv4$ در روتر C برداشته شده است. و بعد از آن روترها از تکنولوژی $IPv6$ ساپورت میکنند و دیتاگرام $IPv6$ ارسال میشود.

(د): ورودی تونل: روتر F

خروجی تونل: روتر C

پروتکل $IPv6$ درون پروتکل $IPv4$ قرار می‌گیرد.

۱.۶

(الف): با توجه به اسلاید ۴ درست صفحه ۶۸، همانطور که میدانید، در الگوریتم *open flow* به جای *forwarding table* یک *flow table* داریم که از ستون‌های *match* و *action* تشکیل شده است. در بخش *match* الگوهای لازم در فیلد‌های هدر پکت برای مچ شدن، بیان شده است.

در بخش *action*، بعد از اینکه بسته با آن الگوها مچ شد، حالا باید طبق *action* باید بگوییم چه اتفاقی برای پکت می‌افتد، مثلاً *drop* یا *forwarding* یا *send* کردن آن به کنترلر رخ دهد.

یک بخش *rule* هم داریم که تا حد ممکن باید دقیق باشد. تا برای موارد مختلف، اتفاقات مختلفی بتواند رخ دهد، مثلاً در این سناریو، ما می‌خواهیم بسته‌های *TCP* و *UDP* که مبدا و مقصد کاملاً یکسانی دارند را از مسیرهای متفاوتی عبور دهیم. اگر قوانین کلی باشند (مثلاً فقط بر اساس *IP* مقصد)، نمی‌توان بین این دو نوع ترافیک تمایز قائل شد. بنابراین باید فیلد *IP Proto* نیز در *Match* لحاظ شود تا تفکیک دقیق انجام گیرد.

(ب): بسته حذف (*Drop*) می‌شود. چون طبق صورت سوال، هر بسته‌ای که با هیچ قانونی در جدول مطابقت نداشته باشد، حذف می‌شود. از آنجایی که ما فقط قوانینی برای ترافیک بین شبکه *s2* و *s3* تعریف کرده‌ایم، روتر *s4* هیچ قانونی برای مدیریت بسته‌هایی که از زیرشبکه خودش، همان (128.122/16) می‌آیند، ندارد.

۲.۶

به جای پورت‌ها ستاره می‌گذاریم یعنی هر چه که پورت ورودی و خروجی هست، همان را می‌گذاریم. داخل پرانتزها در بخش فوروارد اکشن، همان شماره پورت خروجی است. یک نکته که برای درک بهتر جداول از جمینای [۷] کمک گرفتم و آن اینکه در ساختار یک بسته *IPv4*، یک بخش ۸ بیتی به نام *Protocol* وجود دارد. این فیلد مشخص می‌کند که داده‌های موجود در بدنه (*Payload*) بسته از چه نوعی هستند.

اگر مقدار این فیلد ۶ باشد، یعنی بسته حاوی داده‌های *TCP* است.

اگر مقدار این فیلد ۱۷ باشد، یعنی بسته حاوی داده‌های *UDP* است.

(الف): این روتر وظیفه دارد بسته‌های *TCP* دریافتی از *s2* را به سمت *s3* هدایت کند.

<i>Action</i>	<i>Dst Port</i>	<i>Src Port</i>	<i>IP Proto</i>	<i>IP Dst</i>	<i>IP Src</i>
<i>Forward(3)</i>	*	*	6 (<i>TCP</i>)	128.121/16	128.120/16

جدول ۴

(ب): این روتر نقطه شروع است و باید ترافیک را بر اساس پروتکل تفکیک کند.

<i>Action</i>	<i>Dst Port</i>	<i>Src Port</i>	<i>IP Proto</i>	<i>IP Dst</i>	<i>IP Src</i>	<i>Rule</i>
<i>Forward(3)</i>	*	*	6 (<i>TCP</i>)	128.121/16	128.120/16	<i>TCP</i>
<i>Forward(2)</i>	*	*	17 (<i>UDP</i>)	128.121/16	128.120/16	<i>UDP</i>

جدول ۵

(ج): این روتر بسته‌ها را از مسیرهای مختلف دریافت کرده و باید به پورت متصل به زیرشبکه خود (پورت ۱) تحویل دهد.

<i>Action</i>	<i>Dst Port</i>	<i>Src Port</i>	<i>IP Proto</i>	<i>IP Dst</i>	<i>IP Src</i>	<i>Rule</i>
<i>Forward(1)</i>	*	*	6 (<i>TCP</i>)	128.121/16	128.120/16	<i>TCP</i>
<i>Forward(1)</i>	*	*	17 (<i>UDP</i>)	128.121/16	128.120/16	<i>UDP</i>

جدول ۶

۷

بر اساس صورت سوال، پارامترهای اولیه به شرح زیر است:

- اندازه کل دیتاگرام: ۲۴۰۰ بایت
 - اندازه هدر (پیش‌فرض): ۲۰ بایت
 - مقدار داده خالص: $2400 - 20 = 2380$ بایت
 - حداکثر واحد انتقال (*MTU*): ۷۰۰ بایت
 - حداکثر داده در هر تکه: $700 - 20 = 680$ بایت (چون ۶۸۰ بر ۸ بخش‌پذیر است، قابل قبول است)
۱. تقسیم‌بندی حجم داده‌ها

دیتای ۲۳۸۰ بایتی به صورت زیر در بسته‌ها توزیع می‌شود:

$$2380 = 680 + 680 + 680 + 340$$

این یعنی ما به ۴ قطعه (*Fragment*) نیاز داریم

۲. محاسبه آفست (*Offset*)

مقدار آفست بر اساس بایت شروع داده در هر قطعه، تقسیم بر ۸ محاسبه می‌شود

• تکه اول: $0/8 = 0$

• تکه دوم: $680/8 = 85$

• تکه سوم: $(680 + 680)/8 = 170$

• تکه چهارم: $(680 + 680 + 680)/8 = 255$

در تمام بسته‌ها مقدار *Identification (ID)* برابر با ۴۲۲ خواهد بود.

در بسته آخر، *fragflag* را باید برابر صفر بگذاریم یعنی دیگر داده ای برای ارسال نداریم از این نوع.

شماره بسته	طول کل (<i>Total Length</i>)	شناسه (<i>ID</i>)	پرچم (<i>Frag Flag</i>)	آفست (<i>Offset</i>)
بسته اول	700	422	1	0
بسته دوم	700	422	1	85
بسته سوم	700	422	1	170
بسته چهارم	360	422	0	255

جدول ۷: جدول قطعه‌بندی و مقداردهی فیلدهای *ID*، *FF* و *Offset*

۸

راستش این سوال رو جوابش رو توی اسلاید ها پیدا نکردم ولی یک جمله ای از کتاب بود که توضیح خوبی راجع به مشکل پیش آمده برای *NAT* در *P2P* میداد:

«*NAT* در سال‌های اخیر به طور گسترده‌ای مورد استفاده قرار گرفته است. اما *NAT* بدون منتقد (مخالف) نیست. اول اینکه، ممکن است این بحث مطرح شود که شماره پورت‌ها برای آدرس‌دهی به فرآیندها در نظر گرفته شده‌اند، نه برای آدرس‌دهی به میزبان‌ها. این نقض [قاعده] در واقع می‌تواند برای سرورهایی که در شبکه‌های خانگی اجرا می‌شوند مشکل‌ساز شود؛ زیرا همان‌طور که در فصل ۲ دیدیم، فرآیندهای سرور در شماره پورت‌های شناخته‌شده (*well-known*) منتظر درخواست‌های ورودی می‌مانند و همتاها (*peers*) در یک پروتکل *P2P* نیز زمانی که در نقش سرور عمل می‌کنند، نیاز دارند که اتصالات ورودی را بپذیرند. چگونه یک همتا می‌تواند به همتای دیگری که پشت یک سرور *NAT* قرار دارد و دارای یک آدرس *NAT* تخصیص‌یافته توسط *DHCP* است، متصل شود؟ راهکارهای فنی برای این مشکلات شامل ابزارهای عبور از *NAT* (یا *NAT traversal*) است [RFC 5389, RFC 5128].

«[Ford 2005]

برای اطلاعات کامل تر از جمینای کمک گرفتم [۸]

این سه منبع که در کتاب به آن‌ها ارجاع داده شده، ستون‌های اصلی دانش فنی در مورد عبور از NAT هستند. در اینجا خلاصه‌ای از محتوای هر کدام و نقشی که در حل سوال شما (اتصال آلیس و باب) دارند را توضیح می‌دهم:

۱.۸ RFC 5389 (پروتکل STUN)

عبارت STUN مخفف *Session Traversal Utilities for NAT* است.

وظیفه اصلی: این پروتکل به یک کلاینت (مثل آلیس) که پشت NAT قرار دارد اجازه می‌دهد تا آدرس IP عمومی و شماره پورتی را که روتر NAT به او اختصاص داده است، کشف کند. کاربرد در سوال: آلیس و باب با استفاده از STUN می‌فهمند که دنیای خارج آن‌ها را با چه آدرسی می‌بیند. بدون این پروتکل، آن‌ها فقط آدرس‌های داخلی خود (مثل 192.168.1.5) را می‌دانند که برای برقراری ارتباط در اینترنت بی‌فایده است.

۲.۸ RFC 5128 (راهنمای عبور از NAT برای P2P)

این سند یک "بهترین روش" (*Best Current Practice*) برای ارتباطات همتا به همتا است.

محتوا: این RFC انواع مختلف NAT (مخروطی، متقارن و غیره) را تحلیل می‌کند و توضیح می‌دهد که در هر حالت، کدام روش (*Hole Punching* یا *Relaying*) باید استفاده شود. کاربرد در سوال: این سند توضیح می‌دهد که چرا وقتی هر دو طرف پشت NAT هستند، اتصال مستقیم دشوار است و تایید می‌کند که روش *P2P Hole Punching* موثرترین راه برای برقراری ارتباط مستقیم بدون نیاز به تغییر تنظیمات روتر توسط کاربر است.

۳.۸ Ford 2005 (مقاله کلیدی برای Hole Punching)

این ارجاع به مقاله مشهور برایان فورد (*Bryan Ford*) و همکارانش به این عنوان و اسم ارائه شد که "*Peer-to-Peer Communication Across Network Address Translators*" است که در کنفرانس *USENIX* ارائه شد.

اهمیت: این مقاله در واقع "کتاب مقدس" روش *Hole Punching* است. فورد در این تحقیق ثابت کرد که می‌توان از طریق NAT ها، اتصال TCP مستقیم برقرار کرد (کاری که تا قبل از آن بسیار دشوار یا غیرممکن تلقی می‌شد). پاسخ به بخش دوم سوال شما: فورد در این مقاله دقیقاً توضیح می‌دهد که چرا طراحی این روش برای TCP مشکل است:

State Machines: لایه NAT باید وضعیت دست‌تکانی (*Handshake*) پروتکل TCP را دنبال کند و این کار را برای بسته‌هایی که همزمان از دو طرف فرستاده می‌شوند، سخت می‌کند.

Symmetric NAT: او نشان داد که اگر یکی از NAT‌ها از نوع "متقارن" باشد، شماره پورت‌ها مدام تغییر می‌کنند و پیش‌بینی پورت برای "ایجاد حفره" غیرممکن می‌شود.

خلاصه کاربردی برای پاسخ به سوال ۸: اگر بخواهید بر اساس این مراجع پاسخ دهید:

طراحی: باید از روش *TCP Hole Punching* (بر اساس مقاله Ford 2005) و پروتکل STUN (بر اساس RFC 5389) استفاده کنید.

مشکل طراحی: مشکل اصلی در NAT‌های متقارن (*Symmetric*) است (مطابق با تحلیل RFC 5128) که در آن‌ها پورت اختصاص یافته ثابت نیست و باعث شکست فرآیند حفره‌زنی می‌شود، مگر اینکه از یک سرور واسط به عنوان *Relay* استفاده شود.

این هم در ادامه جواب جمینای [۸] بود که به یک سرور واسط و مرکزی برای هندل کردن این موضوع اشاره می‌کرد:

۴.۸ روش پیشنهادی: ایجاد حفره در TCP (*TCP Hole Punching*)

برای اینکه آلیس و باب بدون پیکربندی دستی (مثل *Port Forwarding*) به هم وصل شوند، معمولاً از یک سرور واسط (*Rendezvous Server*) استفاده می‌شود که هر دو به آن دسترسی دارند.

مراحل اجرا:

ثبت‌نام: هم آلیس و هم باب یک اتصال TCP به سرور واسط برقرار می‌کنند.

استخراج اطلاعات: سرور واسط آدرس IP عمومی و شماره پورت (*Public IP : Port*) آلیس و باب را که توسط NAT آن‌ها تخصیص داده شده، یادداشت می‌کند.

تبادل اطلاعات: سرور اطلاعات آلیس را به باب و اطلاعات باب را به آلیس می‌دهد.

اتصال همزمان (*Simultaneous Open*): آلیس و باب به طور همزمان شروع به فرستادن بسته‌های TCP SYN به آدرس عمومی یکدیگر می‌کنند.

وقتی آلیس بسته SYN را به سمت باب می‌فرستد، NAT آلیس فکر می‌کند این یک درخواست خروجی است و یک مسیر برای بازگشت پاسخ باز می‌کند.

بسته آلیس به NAT باب می‌رسد. اگر باب هم دقیقاً در همان زمان بسته SYN خود را فرستاده باشد، NAT باب این بسته ورودی را به عنوان پاسخی به درخواست خروجی باب تلقی کرده و اجازه عبور می‌دهد.

۵.۸ چرا طراحی این روش مشکل است؟ (محدودیت‌ها)

با وجود اینکه تئوری "ایجاد حفره" ساده به نظر می‌رسد، در عمل با مشکلات جدی روبرو است: نوع *NAT* (*Symmetric NAT*): بسیاری از *NAT*‌های سازمانی و مدرن از نوع "متقارن" هستند. در این حالت، *NAT* برای هر مقصد متفاوت، یک پورت خارجی متفاوت اختصاص می‌دهد. یعنی پورتی که آلیس برای اتصال به "سرور واسط" استفاده کرده با پورتی که برای اتصال به "باب" نیاز دارد متفاوت خواهد بود، که باعث شکست فرآیند می‌شود.

مدیریت وضعیت در *TCP*: برخلاف *UDP* که بدون وضعیت (*Stateless*) است، *TCP* یک پروتکل مبتنی بر اتصال و وضعیت است. بسیاری از لایه‌های *NAT* اجازه نمی‌دهند یک پورت که برای "اتصال خروجی" استفاده شده، همزمان برای پذیرش یک "اتصال ورودی" استفاده شود (تداخل در ماشین وضعیت *TCP*). زمان‌بندی (*Timing*): برای موفقیت در *Hole Punching*، بسته‌های *SYN* باید تقریباً همزمان ارسال شوند. اگر بسته یکی زودتر برسد و *NAT* طرف مقابل هنوز "حفره" خروجی را ایجاد نکرده باشد، بسته توسط فایروال *NAT* دور ریخته (*Drop*) می‌شود.

۶.۸ راهکار جایگزین (*Relaying*)

اگر روش *Hole Punching* شکست بخورد (که در حدود ۳۰ تا ۴۰ درصد مواقع به دلیل وجود *Symmetric NAT* اتفاق می‌افتد)، تنها راه باقی‌مانده استفاده از *Relay* (رله) است. در این روش، تمام داده‌های بین آلیس و باب از طریق یک سرور میانی (مثل سرورهای *TURN*) عبور می‌کند. نکته: این روش همیشه کار می‌کند اما باعث افزایش تاخیر (*Latency*) و مصرف پهنای باند سرور می‌شود.

۹

این سوال حذف شد.

۱۰

الف. برای سوییچینگ در یک ساختار گذرگاه مشترک یا همان *bus* حتی اگر دو بسته به پورت‌های متفاوت بروند، همینکه همزمان روی *bus* قرار گرفتند و همزمان ارسال شدند، با هم برخورد و *collision* خواهند داشت. پس، حتی اگر مقصدها متفاوت باشند، بسته‌ها باید به نوبت از باس عبور کنند.

ب. اگر ساختار سوییچینگ به صورت نوار متقاطع و *crossbar* باشد، امکان ارسال همزمان بسته‌ها به چند تا خروجی مختلف هست، چون مسیرهای موازی داریم که با هم تداخل ندارند. (مسیرهای فیزیکی آنها با هم تلاقی نخواهد داشت)

پ. امکان ارسال همزمان دو بسته به یک خروجی در *crossbar* وجود ندارد چون هنوز محدودیت در مقصد را داریم و یک پورت خروجی در هر لحظه فقط می‌تواند یک بسته را ارسال کند. در این حال با ارسال همزمان «تصادم در خروجی» (*Output Contention*) رخ می‌دهد و یکی از بسته‌ها باید در صف منتظر بماند تا انتقال بسته اول تمام شود. [۹]

۱۱

خب برای این سوال، نیاز داریم تا الگوریتم بلمن فورد را در هر نود پیاده‌سازی کنیم. [۱۰]
بعد از تکمیل فایل‌های *node * .c* سعی کردم با ترمینال آن را ران کنم که با توجه به اینکه مک با پردازنده آرم دارم، باید این دستور را ران می‌کردم (طبق چیزی که *chatGPT* گفت [۱۰])

```
(base) alireza@alirezas-MacBook-Air c_linux_code % gcc -std=gnu89 -Wall -Wextra \
-Wno-error=implicit-function-declaration \
-Wno-error=return-type \
-Wno-deprecated-non-prototype \
-o dv distance_vector.c node0.c node1.c node2.c node3.c
./dv
```

مشکل این بود که در *macOS* داریم با یک نسخه مدرن از *Clang* کامپایل می‌کنیم که خیلی از ویژگی‌های قدیمی زبان *C* مثل اعلان ضمنی توابع (*implicit declaration*) یا نداشتن *prototype* برای توابع را به‌عنوان خطا (*error*) در نظر می‌گیرد.

پس کد بالا را برای ران کردن در ترمینال مینویسیم و عددی هم که وارد می‌کنیم وقتی از ما می‌پرسد: Enter TRACE: بهتر است عدد ۲ را وارد کنیم برای دیباگ بهتر که لاگ‌های مربوطه را دهد. چون تریس ۳ جزئیات بیشتری می‌دهد و تریس ۱ هم جزئیات کمتری می‌دهد و همین تریس ۲ مناسب است.
در ضمن گفته شد که *warning* ها را در نظر نگیریم و *ignore* کنیم.
خروجی کد را با تریس ۲ در فایل *output.txt* قرار دادم.
آخرین جدول‌های ساخته شده برای هر نود که بعد از آن دیگر هیچ آپدیتی رخ نداده است را می‌توانید در شکل ۱ ببینید.

```

MAIN: rcv event, t=20002.449, at 3 src: 2, dest: 3, contents:  2  1  0  2

rtupdate3(): received a packet from 2
              via
    D3 |      0      2
    ---|-----
dest 0 |      7      4
    1 |      8      3
    2 |      9      2
MAIN: rcv event, t=20003.146, at 1 src: 2, dest: 1, contents:  2  1  0  2

rtupdate1(): received a packet from 2
              via
    D1 |      0      2      3
    ---|-----
dest 0 |      1      3    999
    2 |      3      1    999
    3 |      5      3    999
MAIN: rcv event, t=20003.852, at 0 src: 3, dest: 0, contents:  4  3  2  0

rtupdate0(): received a packet from 3
              via
    D0 |      1      2      3
    ---|-----
    1 |      1      4     10
dest 2 |      2      3      9
    3 |      4      5      7
MAIN: rcv event, t=20004.211, at 2 src: 3, dest: 2, contents:  4  3  2  0

rtupdate2(): received a packet from 3
              via
    D2 |      0      1      3
    ---|-----
dest 0 |      3      2      6
    1 |      4      1      5
    3 |      7      4      2

Simulator terminated at t=20004.210938, no packets in medium

```

شکل ۱: tables of the bellman ford algorithm for the 4 nodes

توضیح فایل‌های *node * .c*

در فایل‌های *node0.c* تا *node3.c* یک الگوی ثابت پیاده‌سازی شده است و فقط شماره نود و همسایه‌ها و هزینه‌ها تفاوت دارند. در ادامه اجزای اصلی کد به صورت بخش‌بندی شده توضیح داده می‌شود.

(۱) تعریف‌های مشترک ابتدای فایل

`$define INFINITY 999`

عدد 999 به عنوان `INFINITY` در نظر گرفته می‌شود تا نشان دهد مسیر وجود ندارد یا هزینه بسیار زیاد است.

`struct rtpkt`

ساختار `rtpkt` همان بسته‌ای است که نودها برای همسایه‌های مستقیم خود ارسال می‌کنند:

۱. `sourceid` : شناسه فرستنده

۲. `destid` : شناسه گیرنده (باید همسایه مستقیم باشد)

۳. `mincost[4]` : بردار کمترین هزینه‌های فرستنده تا همه مقصدها از 0 تا 3

توابع خارجی `tolayer2` و `creatertpkt`

این توابع داخل شبیه‌ساز `distance - vector.c` تعریف شده‌اند:

۱. `creatertpkt(...)` بسته `rtpkt` را می‌سازد (پر کردن `source/dest/mincost`)

۲. `tolayer2(pkt)` بسته را در شبیه‌ساز به مقصد ارسال می‌کند

(۲) هزینه لینک‌های مستقیم: `connectcostsX[4]`

برای هر نود یک آرایه مانند `connectcosts0[4]` وجود دارد که هزینه لینک مستقیم آن نود تا سایر نودها را مشخص می‌کند. مثلاً برای نود 0:

$$\text{connectcosts0} = \{0, 1, 3, 7\}$$

یعنی هزینه از نود 0 به نود 1 برابر 1، به نود 2 برابر 3 و به نود 3 برابر 7 است.

(۳) جدول فاصله: `dtX.costs[dest][via]`

جدول فاصله به صورت زیر تعریف می‌شود:

$$\text{dtX.costs}[d][v]$$

که معنی آن این است: هزینه رسیدن از نود `X` به مقصد `d` اگر مسیر از همسایه `v` عبور کند.

مثال: `dt0.costs[2][1]` یعنی هزینه رسیدن نود 0 به مقصد 2 با عبور از همسایه 1.

(۴) بردار هزینه مینیمم فعلی: $mincostX[4]$

این آرایه خروجی فعلی الگوریتم است:

$$mincostX[d]$$

یعنی بهترین هزینه‌ای که نود X فعلاً برای رسیدن به مقصد d می‌شناسد. همین بردار به همسایه‌ها ارسال می‌شود.

(۵) ذخیره آخرین بردار همسایه‌ها: $neighbor - mincostX[4][4]$

در این آرایه، آخرین $mincost$ دریافتی از هر همسایه ذخیره می‌شود. این کار به خصوص برای بخش $bonus$ (تابع تغییر هزینه لینک) مفید است تا بتوان ستون‌های جدول را دوباره محاسبه کرد.

(۶) تابع شروع: $rtinitX()$

این تابع در ابتدای اجرای شبیه‌ساز برای هر نود یک بار اجرا می‌شود و کارهای زیر را انجام می‌دهد:

۱. مقداردهی اولیه کل جدول $dtX.costs[][]$ با $INFINITY$

۲. تنظیم هزینه به خود نود برابر ۰

۳. قرار دادن هزینه‌های لینک مستقیم در خانه‌های via مربوطه

۴. مقداردهی اولیه $mincostX[][]$ برابر هزینه‌های مستقیم

۵. ارسال اولین $distance vector$ به همسایه‌های مستقیم

(۷) تابع بروزرسانی: $rtupdateX(rcvdpkt)$

وقتی یک بسته $rtpkt$ از همسایه v دریافت می‌شود:

۱. بردار دریافتی همسایه در $neighbor - mincostX[v][d]$ ذخیره می‌شود

۲. ستون مربوط به $via = v$ در جدول بروزرسانی می‌شود:

$$dtX.costs[d][v] = connectcostsX[v] + neighbor - mincostX[v][d]$$

۳. سپس $mincostX$ دوباره محاسبه می‌شود

۴. اگر تغییری در $mincostX$ رخ داده باشد، نود دوباره بردار جدید را برای همسایه‌های مستقیم ارسال می‌کند

(۸) محاسبه مجدد: $recompute - mincostX()$

این تابع برای هر مقصد d کمترین مقدار در بین همه مسیرهای ممکن را انتخاب می‌کند:

$$mincostX[d] = \min_v \{dtX.costs[d][v]\}$$

اگر مقدار جدید با مقدار قبلی متفاوت باشد، یعنی جدول تغییر کرده و باید $distance\ vector$ دوباره ارسال شود.

(۹) ارسال بردار: $sendmincostX()$

این تابع فقط به همسایه‌های مستقیم نود X بسته می‌فرستد (طبق صورت سوال). برای هر همسایه:

۱. ساخت بسته با $creatertpkt$

۲. ارسال با $tolayer2$

(۱۰) چاپ جدول: $printdtX()$

این تابع برای $debug$ و $trace$ است و جدول فاصله را چاپ می‌کند تا تغییرات قابل مشاهده باشند.

(۱۱) بخش $bonus$: $linkhandler0$ و $linkhandler1$

طبق صورت سوال، تغییر هزینه لینک فقط برای لینک بین نود ۰ و نود ۱ رخ می‌دهد؛ پس فقط $linkhandler0$ و $linkhandler1$ لازم هستند. در این تابع وقتی هزینه لینک تغییر می‌کند:

۱. مقدار $connectcostsX[linkid]$ بروزرسانی می‌شود

۲. ستون $via = linkid$ در جدول مجدداً محاسبه می‌شود

۳. $mincostX$ دوباره محاسبه می‌شود

۴. اگر تغییری رخ داده باشد، بردار جدید به همسایه‌های مستقیم ارسال می‌شود

جمع‌بندی

۱. dt همه مسیرهای ممکن از طریق همسایه‌ها را نگه می‌دارد

۲. $mincost$ بهترین مسیر فعلی برای هر مقصد است

۳. *rtupdate* با دریافت *DV* از همسایه‌ها جدول را بروزرسانی می‌کند

۴. *linkhandler* در صورت تغییر هزینه لینک، ستون مربوطه را اصلاح می‌کند

کد های تکمیل شده به همراه فال ریپورت ضمیمه میشود. خروجی کد هم در فایل output.txt قرار گرفته است و توضیحات کد ها هم به صورت کامنت و هم به صورت متنی در ریپورت موجود است.

مراجع

- [1] “other metrics rather than longest prefix match,” <https://gemini.google.com/share/69bce8db52fe>, 2025.
- [2] “other metrics rather than longest prefix match,” <https://chatgpt.com/share/e/69480742-c0dc-800c-be37-d52ab5af9e10>, 2025.
- [3] “wfq,” <https://gemini.google.com/share/cfdb20d84741>, 2025.
- [4] “public address,” <https://gemini.google.com/share/bf204d86330a>, 2025.
- [5] “nat,” <https://gemini.google.com/share/be0b3c873e3c>, 2025.
- [6] “ipv6,” <https://gemini.google.com/share/aad4a0157785>, 2025.
- [7] “openflow,” <https://gemini.google.com/share/85fa334df0e2>, 2025.
- [8] “nat-in-p2p,” <https://gemini.google.com/share/fc00c0125eeb>, 2025.
- [9] “switching-fabrics,” <https://gemini.google.com/share/91fdf01690cf>, 2025.
- [10] “bellmanford-implementation,” <https://chatgpt.com/share/e/694bcb7f-fe48-800c-a97e-52ad592afb1e>, 2025.